

EXECVIZ VERSION 0 ROADMAP

Interactive code execution visualization — from code input to trace playback

CHART PAPER · 24 × 36 IN · PRINT READY

NORTH STAR

Build a browser experience where a learner writes code, runs it safely, and moves through execution one step at a time while seeing the current line, variables, call stack, and console output.

Version 0 success = one reliable JavaScript/TypeScript trace flow, not three languages with a weak visualizer.

TEAM OWNERSHIP

Akash Bajpai — Frontend Lead

Editor, UI, state management, playback controls, frontend/API integration.

Arush Kumar — Backend Lead

AST instrumentation, sandbox runner, trace contract, execution API, isolation.

Third teammate — Viz/Integration Lead (name TBD)

Timeline, variable/memory rendering, animation model, integration QA.

STATUS LEGEND

- DONE** verified or merged in the repository
- PARTIAL** implemented but incomplete or not end-to-end
- VALIDATE** code exists; lint/build/runtime proof still needed
- PENDING** not implemented in the current main branch

CURRENT REPOSITORY SNAPSHOT

Already present

- Next.js + React + TypeScript app
- Monaco editor and dark UI
- Java/Python/JavaScript legacy output path
- AST parser/instrumenter for JavaScript/TypeScript
- TraceStep, scope, variable, output, and error types
- Node VM sandbox with timeout and step cap
- POST /api/execute returning ordered steps
- Docker execution work merged in local history
- Documentation and setup guides

Not yet end-to-end

- Frontend playback timeline
- Current-line highlighting from returned steps
- Visible variables/memory cards
- Call-stack panel and function transitions
- Real scope values populated in snapshots
- One canonical execution route
- Cross-platform runtime validation
- Automated tests and deployment

PHASED DELIVERY PLAN

Each row is an actionable step. Owner is accountable; All means the team reviews and signs off.

PHASE 0 · FOUNDATION & CONTRACT

F01	Confirm product scope	All	DONE	Lock Version 0 to JavaScript/TypeScript trace playback; Java/Python remain output-only until later.
F02	Repository + branch hygiene	Akash	DONE	Use arushkumar-aiml/ExecViz as the canonical remote; remove accidental files and stale remotes.
F03	Define TraceStep contract	Arush	DONE	Keep `TraceStep`, `ScopeSnapshot`, `VariableSnapshot`, `ExecutionResult`, and error kinds stable.
F04	Choose canonical API	Arush + Akash	PENDING	Remove ambiguity between root app/api, nested backend/app/api, and legacy Express server.
F05	Document MVP acceptance criteria	All	PARTIAL	Write a short checklist: run, trace, inspect, step, reset, error, and safe limits.

PHASE 1 · BACKEND TRACE ENGINE

B01	Parse JavaScript/TypeScript	Arush	DONE	Babel parser accepts JavaScript and TypeScript syntax and returns useful parse errors.
B02	Inject trace hooks	Arush	DONE	Instrument line statements and function entry events with `__execviz_trace`.
B03	Run in VM sandbox	Arush	DONE	Node VM context removes require/process/fs/net and disables string/wasm code generation.
B04	Timeout + step limit	Arush	DONE	5-second default timeout, maximum 5,000 steps, and explicit timeout/step-limit errors.
B05	Capture console events	Arush	DONE	Store output and console trace events with level and serialized arguments.
B06	Capture real scope values	Arush	PENDING	Populate `scopes[].variables` from the executing context; current runner creates empty scopes.
B07	Validate Docker isolation	Arush	VALIDATE	Run the Docker path on Windows and Linux; confirm limits, cleanup, and failure behavior.

PHASE 2 · EXECUTION API & INTEGRATION

I01	Validate request schema	Arush	DONE	Reject empty code, unsupported language, oversized code, and invalid JSON with clear status codes.
I02	Return ExecutionResult	Arush	DONE	Return success, ordered steps, output, error kind/ message, and duration.
I03	Connect frontend fetch	Akash	PARTIAL	Frontend posts code/language, but current UI consumes output and does not consume steps.
I04	Normalize response errors	Akash + Arush	PENDING	Use one response shape and display parse/runtime/ timeout/internal errors consistently.
I05	Cross-platform commands	Arush	VALIDATE	Avoid Unix-only mkdir/rm paths in Windows development; prefer Node APIs or Docker route.
I06	API smoke tests	All	PENDING	Test valid code, syntax error, runtime error, timeout, step cap, empty input, and large input.

PHASE 3 · VISUALIZATION MVP

V01	Create trace state model	Third teammate	PENDING	Store steps, active index, playback status, speed, selected variable, and last error.
V02	Timeline / step list	Third teammate	PENDING	Render every step with kind, source line, function, and selected state.
V03	Current-line highlight	Akash	PENDING	Use Monaco decorations to highlight the active 1-based source line.
V04	Prev / next / reset	Akash	PENDING	Keyboard and button controls; disable boundaries; reset to first step.
V05	Play / pause / speed	Third teammate	PENDING	Automatic playback at 0.5x, 1x, 2x with pause on error or final step.
V06	Variable state cards	Third teammate	PENDING	Show name, value, type, changed/new/removed state, and JSON-safe expansion.
V07	Call-stack panel	Third teammate	PENDING	Show stack frames and function-entry transitions from `callStack`.
V08	Console/ output panel	Akash	PARTIAL	Existing output panel works; make it step-aware and show console events at the active step.

PHASE 4 · QUALITY, SECURITY & DEMO

Q01	Fix lint errors	Akash + Arush	VALIDATE	Remove explicit-any errors, unescaped JSX text, and legacy CommonJS lint blockers without changing behavior.
Q02	Production build	Akash	VALIDATE	Run `npm run build` on the merged main branch after route and UI changes.
Q03	Unit tests	Third teammate	PENDING	Test instrumenter, scope serializer, error mapping, and playback reducer.
Q04	Security review	Arush	PENDING	Threat-model VM and Docker; document demo-only limits and production isolation requirements.
Q05	Responsive/ accessibility pass	Akash	PENDING	Keyboard controls, readable contrast, mobile/tablet layout, loading/error states.
Q06	Demo script + examples	All	PARTIAL	Prepare 3 examples: variables, loop/function call, and error/timeout.
Q07	Deploy preview	Akash	PENDING	Deploy a preview environment with runtime prerequisites and safe execution configuration.

DEPENDENCY CHAIN

B01-B05 Backend trace → B06 Real scopes → V01-V02 Trace state + timeline → V03-V08 Playback UI → Q01-Q07 Demo-ready MVP

WHAT IS PENDING RIGHT NOW

Critical blockers

- Choose one canonical /api/execute implementation.
- Populate actual variables in TraceStep.scopes.
- Build the trace playback UI; current root page remains output-only.
- Resolve the JavaScript/TypeScript versus Java/Python scope for Version 0.
- Run lint, production build, and API smoke tests on the merged Windows checkout.
- Add tests before calling the project demo-ready.

Definition of Done

A user can paste a JavaScript snippet, click Run, see the highlighted current line, move through steps, inspect changed variables and call stack, read output, handle errors, and repeat the run without stale state.

TEAM CHECKPOINTS

Akash — Frontend Lead

Deliver the playback shell, Monaco line decoration, response adapter, and build-clean frontend.

Arush — Backend Lead

Deliver real scope capture, canonical route selection, Docker/VM validation, and security notes.

Third teammate — Viz/Integration Lead

Deliver timeline, variable/memory cards, call-stack view, reducer tests, and end-to-end QA.

All teammates

Review the acceptance checklist, record demo limitations, and approve the final Version 0 tag.

Evidence basis: current main repository audit on 24 September 2026, commit history, source tree, trace contract, instrumenter, sandbox runner, execute route, and frontend page. Team ownership follows the agreed split: Akash = frontend, Arush = backend, third teammate = visualization/integration; third-person name remains TBD.